KEY — Biweekly Assessment BW04 — Weeks 7–8 (Trainer only)

Do not share with students. Excluded from student pack (`*_KEY.md`).

A. MCQ answers

1. **a** — IS-A Staff
2. **a** — Must be the first statement
3. **a** — Same signature as the parent method, runtime object chooses the body
4. **b** — One class
5. **b** — You cannot `new Account(...)`; children fill in abstract methods
6. **b** — An interface (capability / contract)
7. **a** — Same call site, different runtime objects
8. **b** — Folder `campus/wallet/`
9. **b** — Code to the interface (`List`) and construct `ArrayList`
10. **b** — Returns `false` and keeps size the same
11. **c** — `null`
12. **b** — Unboxes `null` → `NullPointerException`

B. Coding — model points

1. Abstract `Account`, two children, `Payable` on the wallet, `List<Account>` loop prints kinds without `instanceof`. Then `Payable p = wallet; p.pay(120);`. Award marks for `abstract` / `implements` / polymorphism. No `throw`.

```
import java.util.ArrayList;
import java.util.List;

public class AccountPolyList {
    public static void main(String[] args) {
        List<Account> accounts = new ArrayList<>();
        accounts.add(new SavingsAccount("Priya", 1000));
        WalletAccount wallet = new WalletAccount("Ravi", 500);
        accounts.add(wallet);
        for (Account a : accounts) {
            System.out.println(a.kind() + " " + a.getHolder());
        }
        Payable p = wallet;
        p.pay(120);
    }
}

interface Payable {
    boolean pay(double amount);
}

abstract class Account {
    private String holder;
    private double balance;

    Account(String holder, double opening) {
```

```
        this.holder = holder;
        this.balance = opening;
    }

    String getHolder() {
        return holder;
    }

    abstract String kind();
}

class SavingsAccount extends Account {
    SavingsAccount(String holder, double opening) {
        super(holder, opening);
    }

    @Override
    String kind() {
        return "savings";
    }
}

class WalletAccount extends Account implements Payable {
    WalletAccount(String holder, double opening) {
        super(holder, opening);
    }

    @Override
    String kind() {
        return "wallet";
    }

    @Override
    public boolean pay(double amount) {
        System.out.println("Paid ₹" + (int) amount);
        return true;
    }
}
```

Expected prints include `savings Priya`, `wallet Ravi`, `Paid ₹120`. (`balance` may sit unused — that is fine if they still declared it private.)

2. Three collections, interface types on the left. Duplicate `P101` prints and returns. Lookup `P999` prints not found (no NPE). `print` + `return` only.

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

public class InventoryLite {
    private List<String> skuOrder = new ArrayList<>();
    private Set<String> knownSkus = new HashSet<>();
    private Map<String, Integer> prices = new HashMap<>();

    void add(String sku, int price) {
        if (price <= 0) {
            System.out.println("price must be positive");
            return;
```

```
        }
        if (knownSkus.contains(sku)) {
            System.out.println("duplicate SKU: " + sku);
            return;
        }
        knownSkus.add(sku);
        skuOrder.add(sku);
        prices.put(sku, price);
    }

    void lookup(String sku) {
        if (!prices.containsKey(sku)) {
            System.out.println("not found: " + sku);
            return;
        }
        System.out.println(prices.get(sku));
    }

    public static void main(String[] args) {
        InventoryLite stock = new InventoryLite();
        stock.add("P101", 40);
        stock.add("P102", 10);
        stock.add("P101", 40);
        stock.lookup("P102");
        stock.lookup("P999");
    }
}
```

Expected: `duplicate SKU: P101`, then `10`, then `not found: P999`. Award marks if they used `containsKey` / Set instead of unboxing a missing `get`.

C. Short answer — model

List when order (and maybe duplicates) matter — cart lines, add-order of SKUs. Set when uniqueness / membership matter — duplicate SKU or email. Map when you look up by id — SKU → price, name → phone. A `HashMap` does not promise receipt order if you loop `keySet()`. Keep a `List` of SKUs for add-order and the `Map` for lookup.

Suggested scoring

Section	Marks
A MCQ (1 each)	12
B Coding (6 each)	12
C Short	6
Total	**30**